
Eduardo Gomes Martins e Lucas de Lima Bergami

Recomendação de Repositórios no GitHub: Uma Abordagem Baseada em Redes Sociais e Aprendizado de Máquina

1 Contexto

Redes sociais modernas vão além de plataformas tradicionais de interação entre pessoas e também aparecem em ambientes colaborativos voltados à produção de conhecimento e desenvolvimento de software. O GitHub é um excelente exemplo desse cenário, pois combina elementos estritamente sociais como seguidores, conexões e construção de comunidades, com elementos de produção colaborativa, representados pelos repositórios de código e pelas contribuições (commits e *pull requests*) realizadas por diferentes indivíduos. Compreender a dinâmica dessa plataforma exige olhar simultaneamente para o código produzido e para as relações humanas que o sustentam.

2 Objetivo

Neste projeto, propõe-se o desenvolvimento de um sistema de recomendação de repositórios baseado em análise de redes sociais e aprendizado de máquina. O objetivo central é estimar a probabilidade de que um determinado usuário venha a contribuir em um repositório específico que ainda não faz parte do seu histórico de colaboração.

Em outras palavras, busca-se transformar o problema de predição de arestas em uma tarefa de recomendação de conteúdo, na qual os repositórios desempenham o papel dos itens recomendados para os desenvolvedores. Além do objetivo prático de recomendação, o trabalho almeja investigar como as relações sociais prévias, a proximidade estrutural na rede e as características intrínsecas dos projetos influenciam a formação de novas conexões dentro de uma comunidade real de desenvolvimento colaborativo.

3 Metodologia

3.1 Parte 1: Descrição dos dados e modelagem

A base de dados utilizada neste trabalho foi coletada diretamente da API pública do GitHub. Foram extraídos dados detalhados que compõem tanto o perfil dos desenvolvedores (contagem de seguidores, número de eventos, idade da conta, linguagens mais utilizadas) quanto características dos repositórios (tamanho, estrelas recebidas, issues abertas e arquivos *README*). Ao final da coleta, obteve-se um conjunto consolidado totalizando pouco mais de 1.200 usuários únicos e cerca de 5.000 repositórios mapeados. O processo de extração não envolveu SGBDs complexos; as respostas da API no formato JSON foram limpas e estruturadas diretamente em formatos tabulares (CSV) e arquivos de grafos. Do escopo dos dados, foram excluídas as interações a

nível de código-fonte em si (o conteúdo dos *commits*), mantendo-se apenas o registro de que a contribuição existiu.

A modelagem da rede foi concebida como um ecossistema heterogêneo. O grafo construído possui dois tipos de vértices (nós): Usuários e Repositórios. As conexões (arestas) dessa rede representam três relações observáveis distintas:

- **Segue:** Aresta direcionada de um Usuário para outro Usuário.
- **Possui:** Aresta direcionada de um Usuário para um Repositório do qual ele é dono.
- **Contribui:** Aresta direcionada de um Usuário para um Repositório de terceiros, extraída do *endpoint* de contribuidores.

Optou-se por implementar um *multigrafo* dirigido, em vez de um grafo simples. Essa escolha ocorreu porque um mesmo par usuário-repositório pode abrigar mais de uma relação simultaneamente — o caso mais comum é o criador de um projeto aparecer tanto como dono (possui) quanto como colaborador (contribui) de seu repositório, e um grafo simples acarretaria na perda de uma dessas informações.

3.2 Parte 2: Experimentos

3.2.1 Descrição de Características e Modelagem de Texto

Para alimentar a recomendação, foi concebido um módulo de engenharia de características. Para cada par possível composto por um usuário u e um repositório candidato r , são extraídas dezenas de métricas brutas dos nós. Somado a isso, também são processadas as propriedades relacionais descritas na Tabela 1.

Tabela 1: Características relacionais primárias usadas para descrever o par usuário-repositório.

Característica	Origem nos dados	O que mede
text_similarity	<i>README</i> de perfil do usuário e <i>README</i> do repositório	Similaridade de cosseno entre os vetores <i>TF-IDF</i> dos dois textos
language_match	Linguagens do usuário e do repositório	Coefficiente de Jaccard entre os dois conjuntos de linguagens
follows_owner	Grafo de relações “segue”	Indica se o usuário segue o dono do repositório
common_contributors_following	Grafo de relações “segue” e “contribui”	Quantos contribuidores do repositório já são seguidos pelo usuário
shared_repos_with_owner	Relações “possui” e “contribui”	Em quantos outros repositórios o usuário e o dono já apareceram juntos
graph_proximity	Grafo completo (usuários e repositórios)	$1/(1+d)$, em que d é a distância (arestas) no grafo não dirigido
repo_popularity	Estrelas, <i>forks</i> e <i>watchers</i> do repositório	Logaritmo da soma dessas três contagens

A característica textual (*text_similarity*) utilizou a vetorização pelo método *TF-IDF* sobre o corpo de texto dos repositórios e perfis pessoais para capturar a interseção de interesses semânticos.

3.2.2 Recomendação de Conteúdo (Modelagem)

A camada de recomendação baseou-se em modelagem supervisionada de Machine Learning. O rótulo positivo ($y = 1$) foi definido pela presença de uma aresta de “contribui”. Já os exemplos negativos ($y = 0$) necessitaram de amostragem inteligente, mesclando negativos “difíceis” (repositórios de pessoas que o usuário já segue, mas não contribui) com negativos aleatórios do restante da base, forçando o modelo a ser rigoroso. A divisão treino e teste ocorreu a nível de usuários (*GroupShuffleSplit*), e não por pares, evitando que o modelo fosse testado em pessoas com as quais já havia treinado parcialmente.

O classificador principal utilizado foi o *XGBoost*, um modelo de *gradient boosting* sobre árvores de decisão. Tal algoritmo foi o escolhido porque nossa matriz consolidada abriga variáveis vastamente heterogêneas (binários, contagens absolutas, similaridades percentuais e cálculos topológicos de grafo), cenário no qual árvores de decisão superam outros modelos sem a necessidade de normalizações agressivas de dados.

4 Discussão

Para avaliar o funcionamento do sistema em um contexto real, os modelos foram submetidos ao processo de validação utilizando o classificador supervisionado XGBoost treinado. Inicialmente, durante a experimentação técnica, o modelo atingiu desempenho estatístico próximo de perfeito nas métricas de curva ROC. Porém, uma análise topológica constatou um vazamento de dados: a própria aresta de contribuição estava alimentando o cálculo de proximidade estrutural (`graph_proximity`). Após isolarmos a aresta avaliada do restante da rede no ato de treino, o sistema assumiu um caráter realista e consistente, balanceando as características da rede em vez de superestimar a distância pura.

Foi conduzido um estudo de caso qualitativo para entender de perto a ordenação gerada pela IA, submetendo os perfis dos usuários `luccastera` e `mgrdcm` à inferência do modelo supervisionado treinado, como documentado nas capturas do terminal das Figuras 1 e 2.

```
[info] usando modelo treinado: ./data/model/contrib_model.joblib
```

Top 10 recomendações para 'luccastera':

repo	score	text_similarity	language_match	follows_owner	common_contributors_following	shared_repos_with_owner	graph_proximity	repo_popularity
thomasjo/maturin	0.739143	0.0	0.363636	0.0	0	0	0.25	0.0
pielez/docuusal	0.739143	0.0	0.363636	0.0	0	0	0.25	0.0
aaronmfraser/pagerbeauty	0.738683	0.0	0.400000	0.0	0	0	0.25	0.0
AdamTreineke/lwc-recipes	0.738683	0.0	0.400000	0.0	0	0	0.25	0.0
beanieboi/weave	0.738683	0.0	0.400000	0.0	0	0	0.25	0.0
markbate/aws-amplify-quick-notes	0.738683	0.0	0.375000	0.0	0	0	0.25	0.0
camilopayan/setup-ruby	0.738683	0.0	0.375000	0.0	0	0	0.25	0.0
caius/isbupsefevet.com	0.738683	0.0	0.375000	0.0	0	0	0.25	0.0
nerztalk/hodky	0.738683	0.0	0.400000	0.0	0	0	0.25	0.0
collin/mp4box.js	0.738683	0.0	0.375000	0.0	0	0	0.25	0.0

Figura 1: Top 10 recomendações utilizando o modelo supervisionado XGBoost para o usuário `luccastera`.

Ao examinar as recomendações de `luccastera`, constata-se que o classificador atribuiu probabilidades finais muito próximas para os candidatos de ponta, orbitando de forma densa em torno de um *score* de aproximadamente 0.739. Esse fenômeno, longe de ser uma mera coincidência matemática, aponta para a tendência do algoritmo em agrupar projetos estruturalmente irmãos dentro das mesmas partições da árvore de decisão.

As métricas subjacentes do modelo indicaram o porquê dessas notas uniformes: o comportamento preditivo foi fortemente dominado por indicadores sociotécnicos. A variável `follows_owner` foi responsável por quase 29,9% da importância preditiva, mostrando que a ponte direta criador-colaborador é o maior combustível para o engajamento de novos projetos na plataforma. Outras variáveis estruturais figuraram com alto impacto, como a natureza do repositório ser derivativa (`repo_is_fork`, 15,2%) e a proximidade estrutural entre o recomendando e o projeto (`graph_proximity`, 12,5%).

Em contrapartida, as informações estatísticas absolutas sobre os perfis, como o simples ato de um usuário ter muitos seguidores, possuir blog ou tuitar sobre tecnologia tiveram impacto preditivo marginalizado, provando que um ecossistema produtivo de nicho reage melhor às conexões interligadas do que à simples popularidade nominal do usuário.

```
[info] usando modelo treino: ./data/model/contrib_model.joblib
```

Top 10 recomendações para 'mgrdcm':

repo	score	text_similarity	language_match	follows_owner	common_contributors_following	shared_repos_with_owner	graph_proximity	repo_popularity
simmani5/rustdesk-server	0.725191	0.0	0.444444	0.0	0	0	0.25	0.000000
anderson/meilisearch-rails	0.724018	0.0	0.428571	0.0	0	0	0.25	0.000000
YuzeHao2023/webpage	0.724018	0.0	0.428571	0.0	0	0	0.25	0.000000
marcospereira/httpbin	0.724018	0.0	0.428571	0.0	0	0	0.25	0.000000
dmitytrager/upright	0.724018	0.0	0.428571	0.0	0	0	0.25	0.000000
lucastera/view_component	0.721660	0.0	0.444444	0.0	0	0	0.25	0.000000
alloy/fluentui-contrib	0.721660	0.0	0.428571	0.0	0	0	0.25	0.000000
qichunren/mission_control-jobs	0.721660	0.0	0.428571	0.0	0	0	0.25	0.000000
AdamReineke/utam-js-recipes	0.721166	0.0	0.500000	0.0	0	0	0.25	0.000000
Ledermann/cypress-rails	0.720082	0.0	0.428571	0.0	0	0	0.25	1.078612

Figura 2: Top 10 recomendações utilizando o modelo supervisionado XGBoost para o usuário mgrdcm.

A submissão do usuário mgrdcm (Figura 2) consolidou esse padrão. Mais uma vez, observou-se uma variação compacta de pontuações (0.720 a 0.725) atrelada a uma afinidade linguística (language_match entre 0.43 e 0.50) e proximidades estruturais imutáveis de 0.25.

Um achado de extrema relevância nestas validações é a persistente anulação do parâmetro text_similarity. A esmagadora maioria dos candidatos propostos apresentava índice zero nessa similaridade de texto. Como grande parte da comunidade de código aberto não investe esforços contínuos na manutenção detalhada de seus READMEs de perfil pessoal, a inteligência artificial percebeu a alta esparsidade e o ruído desse campo, delegando a responsabilidade de recomendação primária inteiramente ao mapeamento do "quem conhece quem" (social) e do "quem interage com o quê" (topológico). Isso reforça uma visão ampla sobre a engenharia de software contemporânea: antes de focar na retórica dos perfis, os profissionais interagem em "bolhas" orientadas pela comunidade, confiando fortemente na indicação silenciosa gerada pelas conexões que eles seguem e com quem dividem projetos em comum.

5 Conclusões

Neste trabalho, demonstrou-se a viabilidade de construir um sistema de recomendação de repositórios no ecossistema do GitHub por meio da junção de características do desenvolvedor, propriedades de código e a extração de métricas sobre um grafo heterogêneo de milhares de nós. Identificamos com êxito que as interações diretas (seguir criadores) e fatores estruturais possuem muito mais peso na decisão de um programador em contribuir em um projeto alheio do que a afinidade por descrições textuais isoladas ou os atributos puramente de vaidade do perfil.

No que tange às limitações do que não pôde ser feito com total excelência técnica, a exploração semântica com matrizes TF-IDF (vetorização dos perfis de desenvolvedores) mostrou-se ineficaz, dado o fato concreto de que grande parcela dos usuários opta por não manter descrições pessoais extensas para serem analisadas matematicamente. Além disso, a atual modelagem da rede foi obrigada a operar de maneira estática (instantâneo do cenário de coleta).

Como caminhos lógicos de continuação para esse trabalho, a pesquisa clama pela introdução de grafos temporais, em que cada aresta passe a obedecer a cronologia das interações e consiga delinear ciclos de alta e baixa popularidade para um código. O aprofundamento da extração topológica via métodos modernos de redes neurais, adotando embutimentos vetorizados de grafos completos (GNNs ou Node2vec), figura como o próximo salto arquitetural necessário para transcender a simples engenharia manual de características rumo ao verdadeiro aprendizado de relações invisíveis no ecossistema de software livre.